Bitty Vault: Self Custody That Survives a Stolen Key

Abstract. In self custody, whoever holds the key can take everything, instantly. We propose a vault that assumes the key will leak, and limits what it can do. Safer changes apply instantly. Riskier ones wait in public, behind a timelock the owner chose. What can leave without waiting is capped. One transaction locks the vault and voids all a thief has queued. A stolen key then costs a bounded sum per period, if the owner notices in time.

---

1. Introduction

A wallet trusts any valid signature. A payment to a friend and a transfer of the whole balance to a thief look the same to it. Both take effect instantly. A multi-signature wallet needs more keys, but the rule is the same: once enough keys sign, all is gone.

Banks work another way. A new payee takes time to set up. Large sums are delayed. The owner can freeze the account. None of this stops a thief from acting. It stops a thief from acting fast.

Decentralized finance has a tool for the same problem. A protocol that holds other people's money puts each change to its rules behind a *timelock*. The change is made public, and must wait before it runs. It does not stop a bad change. It makes a bad change slow and visible.

There, the timelock guards users from those who run the protocol. We point it at the owner's own key. The vault cannot tell the owner from a thief who holds that key, and does not try.

2. Architecture

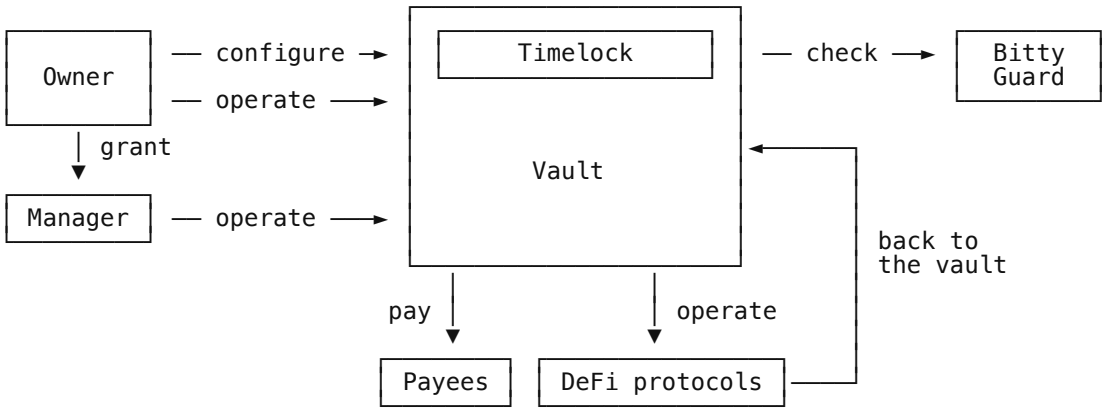


Figure 1: architecture

- The **owner** sets the rules and operates the vault, and alone can move value out. Most often this is one person with one key. It may also be a multi-signature wallet.
- The **manager** is an optional agent the owner appoints, a person or a program. It can operate the vault, but never send value out or change the rules. Its grant always ends.

- The **Bitty Guard** is the registry all vaults check, maintained by Bitty (§4).

Each wallet has one vault, at an address worked out from the wallet's own. It is known before the vault exists, so the vault can be funded first and made later. It never changes, even when the owner does.

3. Timelock

Every change lowering risk applies instantly; raising risk waits a delay **T** that the owner picks.

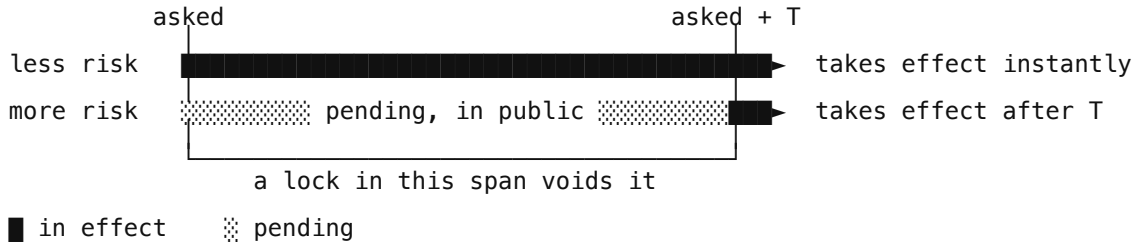


Figure 2: two speeds

Shortening T is itself a rise in risk, so it waits out the current T. A thief gains nothing by trying.

A protocol's timelock delays every change. The vault's delays only the risky ones, so the owner is never slowed down in protecting themselves.

The design asks one thing of the owner: notice within T. Every risky change is public from the moment it is asked for, and a service can watch the vault and warn the owner of any change they did not make.

4. Bitty Guard

The Bitty Guard keeps an allow list of assets and DeFi protocols, so a vault cannot use scam assets or protocols.

Bitty runs the guard like a vault: in public, with delay. It follows the successful on-chain governance of DeFi protocols: proposals are voted on, and a passed one waits before it runs. A **guardian** can cancel a proposal, and can act instantly, but only to restrict.

Governance cannot reach into a vault. It can list new code, but only the owner installs it. It can drop a protocol, but value there can always come back.

5. Payees

A **payee** is an address the owner approved in advance, such as their own exchange account. Payments to a payee are not capped. This is how the owner moves large sums, and anything that cannot be priced.

A new payee can be paid only after T. Removing a payee waits T too, so a thief cannot strike out the owner's safe addresses before the lock.

6. Allowances

Some value may leave without waiting. Two allowances cap it, per period:

- **S**, what may be sent to an address that is not a payee;
- **L**, what trades may lose against an oracle price.

Zero turns either off.

A new vault can be designed to start careful: S at zero, L low.

7. Vault Locks

The delay gives the owner time. The lock is what they do with it.

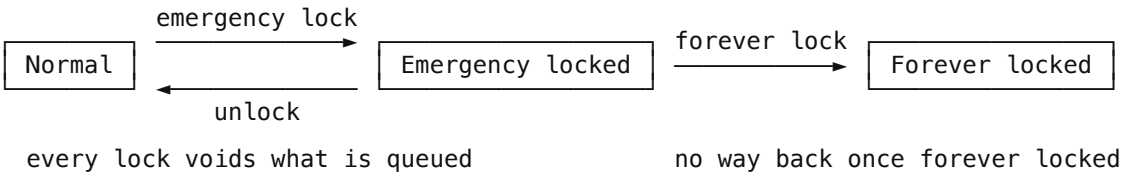


Figure 3: locks

A lock is one transaction, and takes effect instantly. It voids every risky change still pending. While locked, value leaves only to payees approved before the lock. Nothing can be loosened, and the only trade left is a sale for a stablecoin, still within L.

A thief with the key can unlock. That brings nothing back: all they queued is void, and must wait T again. The owner can lock again at any time, and the thief can never void what the owner set up.

A **forever lock** is the same, for good. It needs a payee that can be paid at that moment, so a vault is never locked with no way out.

A lock voids the whole queue at a fixed cost, however much a thief queued.

8. Conclusion

We have proposed a vault that survives a stolen key. Risky changes are slow and public, and what is fast is capped. Payees give the owner a way out a thief cannot close, and a lock voids what a thief queued. Safety no longer rests on the key staying secret, but on the owner noticing in time, within a window they chose.